

Foodie Express - Interview Q&A

Bhushan Mahajan (Fresher, Full-Stack) | github.com/Bhushan2602/Foodie-Express-Project

React 19 + Spring Boot 3.2 + Gateway + JWT + PostgreSQL + MongoDB + Redis + Docker + Razorpay

****Candidate:**** Bhushan Mahajan (Fresher, Full-Stack) | ****Repo:**** github.com/Bhushan2602/Foodie-Express-Project

****Stack:**** React 19 + Spring Boot 3.2 + Spring Cloud Gateway + JWT + PostgreSQL + MongoDB + Redis + Docker + Razorpay

1. Project Overview

****Q1. What is Foodie Express?****

A Swiggy/Zomato-style food delivery platform: 4 Spring Boot microservices behind a Cloud Gateway, React frontend, 45+ seeded restaurants across 13 cities, Razorpay payments (demo mode), admin analytics, restaurant-owner and delivery-partner dashboards, promo engine, order tracking, scheduled delivery.

****Q2. Draw the architecture in words.****

React (Vite) -> Spring Cloud Gateway (:8080) -> user-service (:8081, Postgres foodie_users), restaurant-service (:8082, Mongo foodiedb + Redis cache), order-service (:8083, Postgres foodie_orders). Infra in Docker: Postgres, Mongo, Redis, Kafka, Zookeeper, ES, Kafka UI. Gateway routes by path (`/api/auth/\*\*`, `/api/restaurants/\*\*`, `/api/orders/\*\*`, `/api/payments/\*\*`).

****Q3. Why microservices instead of a monolith?****

Independent scaling (restaurant reads scale separately from orders), independent databases per bounded context, independent deploys. Tradeoff I accept: more ops complexity - handled with one `docker compose up --build`.

****Q4. What does each service own?****

User-service: registration, login, JWT, BCrypt, roles. Restaurant-service: restaurant/menu CRUD, seeding, Redis caching. Order-service: orders, promo validation, Razorpay verify, delivery state machine. Gateway: routing, CORS, actuator.

****Q5. How do services communicate?****

Synchronously via REST through the gateway. No service-to-service calls currently - each owns its data (DB-per-service). Async (Kafka events) is the planned next step; the broker already runs in Compose.

2. Spring Boot / Microservices

****Q6. How does a request flow through the gateway?****

Client -> gateway route predicate matches path -> forwarded to service URI from env (`USER\_SERVICE\_URL` etc.) -> service validates JWT (user-service filter) -> hits DB. CORS is centralized in gateway `application.yml`, so services stay clean.

****Q7. Walk me through the JWT flow.****

Login posts email/password -> `UserService.loginUser` checks BCrypt -> `JwtUtil.generateToken(email, role)` -> frontend stores response in `localStorage` (`foodie\_user` + `token`) -> Axios interceptor attaches `Bearer` header -> `JwtAuthenticationFilter` validates before the security chain. Stateless sessions.

****Q8. How is role-based access enforced?****

`SecurityConfig` permits `/api/auth/\*\*` + Swagger/health publicly, restricts `/api/orders/all/\*\*` to `ROLE\_ADMIN`, delivery

routes to ``ROLE_DELIVERY_PARTNER/ADMIN``, restaurant routes to ``ROLE_RESTAURANT_OWNER/ADMIN``. Frontend mirrors with ``AdminRoute/CustomerRoute/DeliveryPartnerRoute/RestaurantOwnerRoute``.

****Q9. You blocked admin self-registration - how?****

``AuthController.register`` uses an allowlist: only ``ROLE_USER``, ``ROLE_DELIVERY_PARTNER``, ``ROLE_RESTAURANT_OWNER`` accepted from the client; anything else (including ``ROLE_ADMIN``) falls back to ``ROLE_USER``. Admins are promoted directly in Postgres (``UPDATE users SET role='ROLE_ADMIN'...``).

****Q10. How does promo validation work? Why server-side?****

``PromoService`` holds the rules (WELCOME50 flat-50 min-199 first-order-only; FOODIE20 20%-up-to-150; FREEDEL free delivery; HUNGRY30 flat-30). Frontend calls ``POST /api/orders/promos/validate`` for instant feedback, but ``placeOrder`` **recomputes** delivery fee (Rs. 40 × restaurant count, max 5) + 5% tax and re-validates the promo, rejecting forged discounts (>Rs. 1 drift). Client-only promos are trivially tampered, so the server is authoritative.

****Q11. Why is WELCOME50 first-order-only, and how is it enforced?****

Business rule: acquisition coupon. Enforced by querying ``OrderRepository.findBy userEmail`` - if history exists, both ``/validate`` and ``placeOrder`` reject with "valid on your first order only".

****Q12. Explain the delivery state machine.****

``PENDING/PREPARING`` (kitchen) -> owner ``Broadcast to All`` -> ``READY`` (unassigned, live in every partner's pool) -> first partner to ``Accept`` wins -> ``ON THE WAY`` -> ``DELIVERED``. ``Decline`` appends to a multi-email ``declinedBy`` list and stays ``READY``. ``Cancel`` (customer) allowed before ``DELIVERED`` with fee policy. Invalid transitions throw, surfaced by ``GlobalExceptionHandler``.

****Q13. Broadcast vs single-assign - why broadcast?****

First version randomly assigned one partner (could re-pick a decliner - decline loop). Broadcast shows the order in every partner's Open Pool; fastest accept wins; late acceptors get "just accepted by another partner". If all decline, the owner sees ``? All declined`` with ``declined/total`` counts and one-tap rebroadcast (clears decline history).

****Q14. What is the cancel fee policy?****

Free in ``PENDING/PREPARING``; Rs. 30 once a partner is involved (``READY/ON THE WAY``); ``DELIVERED`` can't cancel. Stored as ``cancellationFee`` on the order; COD keeps ``UNPAID``, online is refund-minus-fee (recorded; real refund via Razorpay dashboard in production).

****Q15. What do Actuator and Swagger give you?****

``/actuator/health`` per service (used in Compose healthchecks + interview demo), ``/swagger-ui.html`` per service for live API docs. Added via ``spring-boot-starter-actuator`` + ``springdoc-openapi`` 2.3.0.

3. Data & Messaging

****Q16. Why Postgres for orders/users but MongoDB for restaurants?****

Orders/users are relational (joins, transactions, uniqueness on email). Restaurants are document-shaped (nested menus, flexible fields like rating/meta) - Mongo fits, and seeding 45 restaurants with menus is natural as documents.

****Q17. How does Redis caching work?****

``restaurant-service`` has ``@EnableCaching`` + ``CacheConfig`` (10-min TTL, JSON serializer). ``getAllRestaurants`` -> ``restaurants`` cache; ``getRestaurantsByCity`` -> ``restaurantsByCity`` keyed by lowercase city. All writes ``@CacheEvict`` both caches. Backed by the Redis container (``SPRING_REDIS_HOST=redis`` in Compose).

****Q18. Is Kafka actually used? Be honest.****

Honestly: the broker + Zookeeper + Kafka UI run in Compose, but no producer/consumer is wired yet - order events are my planned next step (publish `order-created`, notification consumer). I never claim otherwise; the README marks it "planned".

****Q19. Same question for Elasticsearch?****

Same honest answer: container runs, no indexing wired yet. Search today is backend filtering (city/cuisine/dish match) + frontend filters. ES full-text is roadmap.

****Q20. How do you handle validation errors consistently?****

Bean Validation (`@Valid`, `@NotBlank`, `@Email`, `@Size`) on DTOs + `GlobalExceptionHandler` (`@RestControllerAdvice`) returning `{timestamp, status, error, message}` JSON. Frontend `api.js` reads `error.response.data.message` for toasts.

****Q21. How did you add columns like promo/rating without breaking seeds?****

Added nullable fields with defaults + backward-compatible constructors (e.g. 7-arg `Restaurant`, 5-arg `MenuItem`), and `DataInitializer` backfills rating/reviews/delivery-time/cost-for-two/dish-images deterministically. Old Mongo docs fall back to frontend-computed meta, so no migration downtime in demo.

4. Security

****Q22. How are passwords stored?****

BCrypt via `BCryptPasswordEncoder` - never plaintext. Login compares with `matches()`.

****Q23. What stops someone forging a promo discount?****

`placeOrder` recomputes the discount server-side from `itemTotal` + delivery fee and compares with the client-sent `discountAmount`; drift >Rs. 1 throws "discount mismatch".

****Q24. JWT in localStorage - isn't that XSS-vulnerable?****

Yes, and I document it in the README as a demo tradeoff. Production fix: httpOnly cookies + refresh-token rotation + strict CSP. I chose localStorage for simplicity across the Vite + gateway setup.

****Q25. How is CORS handled?****

Centralized in gateway (`CORS\_ORIGINS` env, defaults to Vite + Docker UI origins). Services don't duplicate CORS config.

****Q26. What about secrets in git?****

`.env` is gitignored; only `.env.example` with placeholder values is committed. Compose reads `\${POSTGRES\_PASSWORD}`, `\${JWT\_SECRET}` etc. with safe local defaults.

5. React Frontend

****Q27. How do carts stay per-user?****

Cart key is `foodie\_cart\_<email>` (or `foodie\_cart\_guest`). `AuthContext` fires a `foodie-auth-change` window event on login/logout; `CartContext` reloads from the right key. Legacy shared `foodie\_cart` is migrated once then deleted - that was the original cross-user leak bug.

****Q28. How do you stop admins from ordering?****

Defense in depth: `addToCart` checks the stored role and toasts; UI shows "View only / Staff preview" instead of ADD buttons; cart icon/floating bar/quick-add hidden; `/cart` route guarded by `CustomerRoute` (ROLE_USER only).

****Q29. Why is the initial load fast despite Recharts + Framer Motion?****

Route-level `React.lazy()` (14 pages) + `Suspense` skeleton fallback + Vite 8 `advancedChunks` splitting `vendor-react/motion/charts`. Initial chunk ~250KB instead of one 970KB bundle; charts load only on dashboard routes.

****Q30. What is the Ctrl+K palette?****

`SearchPalette.jsx`: global keyboard shortcut opening a command-palette search with debounced live results -> restaurant page, or Enter to `/explore?q=`. Single global search lives in the navbar; the old duplicate hero search was removed.

****Q31. Explain the two themes.****

`Sunset` (default orange enterprise) vs `Liquid Glass` (iOS-style translucent saturated cards, gradient buttons) via `documentElement.dataset.theme` + CSS variables, persisted as `foodie\_ui\_theme`. Separate from light/dark mode (`foodie\_theme`). Switcher in navbar (desktop + mobile drawer).

****Q32. What are the 3D cards?****

`lift-3d` utility: hover lifts, tilts 2°, zooms image, themed shadow - applied to Home/Explore/Featured cards in both themes with a fluid easing variable.

****Q33. Polling vs WebSocket - why poll?****

Order tracking polls every 5s, partner pool every 8s, owner every 5s. Simple and demo-reliable; WebSocket/STOMP push is the documented next step for tracking.

6. Docker / DevOps

****Q34. What's in the Compose stack?****

Postgres, Mongo, Redis, Zookeeper, Kafka, Kafka UI, Elasticsearch + 4 Java services + UI (nginx) - 12 containers, one `up --build`. Postgres healthcheck gates Java services; Mongo got `start\_period: 40s` after slow-machine failures.

****Q35. How does CI work?****

`.github/workflows/build.yml`: matrix build of all 4 services (`mvnw package`) + frontend (`npm ci && npm run build`) on push/PR with Java 21 + Node 20.

****Q36. How do envs flow into containers?****

Compose interpolates `.env` (`POSTGRES\_\*`, `JWT\_SECRET`, `RAZORPAY\_\*`, `CORS\_ORIGINS`) into service environment; Spring relaxed binding maps `SPRING\_\*` to `spring.\*` yml keys with localhost defaults for local runs.

****Q37..env got out of sync with the DB volume once - what happened?****

Changed `POSTGRES\_PASSWORD` after first `up`; the volume kept the old password -> `FATAL: password authentication failed` -> Hibernate "Unable to determine Dialect". Fixed with `down -v` reseed (dev only). Lesson: don't rotate DB passwords without volume reset in dev.

7. Payments

****Q38. How does Razorpay integrate?****

Frontend loads `checkout.js`, backend `PaymentService.create-order` returns a Razorpay order (or `demo\_\*` order

when no keys configured), frontend collects payment, `placeOrder` sends ids + signature, backend `verifyPayment` checks the signature before saving as `PAID`. Demo mode lets the full flow run without real keys.

****Q39. COD vs online in the order lifecycle?****

`cod` -> `PREPARING`/`PENDING` + `UNPAID`; online -> signature verify -> `PAID` + `PREPARING`. Cancel keeps the distinction (COD: fee recorded; online: refund-minus-fee).

****Q40. Where does the tip/delivery-slot go?****

Cart collects tip + Now/Later (+ datetime picker, validated ?30min ahead); carried via `sessionStorage` to Payment, stored as `deliverySlot/scheduledFor` on the order, shown as ? badges in Payment + owner/partner dashboards.

8. HR / Fresher Traps

****Q41. What was the toughest bug and how did you debug it?****

The shared-cart leak: one `localStorage` key across users. Reproduced by logging in as admin then customer, inspected keys in DevTools, fixed with per-user keys + auth-change event. Lesson: scope client state by identity.

****Q42. How would you scale this 10x?****

Read replicas + Redis for restaurant reads, Kafka for order/notification events, gateway rate limiting, DB indexes on email/city/status, frontend virtualization. Bottleneck today: unpaginated restaurant fetches + polling.

****Q43. What would you rewrite with more time?****

Kafka eventing first (removes polling), then real ES search, then refresh-token auth with httpOnly cookies.

****Q44. Did you use AI help?****

Yes for scaffolding/review, but I can explain every line: I debugged the Dialect/auth failure, the brace mismatches, the Vite 8 chunking API change (`manualChunks` -> `advancedChunks`) hands-on.

****Q45. Why should we hire a fresher with one big project over many small ones?****

One production-shaped system forced me through auth, state machines, caching, pricing integrity, Docker, CI, and real user-role bugs - the exact surface area of junior backend/full-stack work - instead of tutorial-depth breadth.

Generated for interview prep. Honest scope: Kafka/ES run in Docker but are not wired to app logic yet.